

- Describe piecewise linear functions
- Expressive enough to approximate complex relationships between multi-dimensional inputs & outputs.

3.1: Neural Network Example:

- They are functions $y = f[x, \theta]$ that map multivariate inputs x to multivariate outputs y .

- Example network:

$$y = f[x, \theta] \rightarrow \text{Represents a family of functions}$$

$$= \theta_0 + \theta_1 a[\theta_{10} + \theta_{11}x] + \theta_2 a[\theta_{20} + \theta_{21}x] + \theta_3 a[\theta_{30} + \theta_{31}x]$$

↳ This model has 10 parameters:

$$\theta_0, \theta_1, \theta_2, \theta_3, \theta_{10}, \theta_{11}, \theta_{20}, \theta_{21}, \theta_{30}, \theta_{31}$$

The model has 3 parts:

- 1) Compute 3 linear functions of the input data

$$\theta_{10} + \theta_{11}x, \theta_{20} + \theta_{21}x, \theta_{30} + \theta_{31}x$$

- 2) Pass these results to an activation function

$$a[\cdot]$$

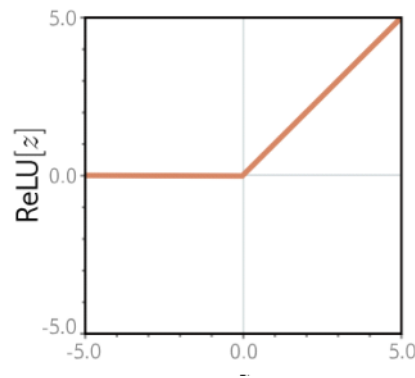
- 3) Weight the activations with $\theta_1, \theta_2, \theta_3$ & offset with θ_0

- There are many choices for the activation function. Most common is RELU:

$$a[z] = \text{ReLU}[z] = \begin{cases} 0 & z < 0 \\ z & z \geq 0 \end{cases}$$

↓
stands for Rectified
linear unit

Figure 3.1 Rectified linear unit (ReLU). This activation function returns zero if the input is less than zero and returns the input unchanged otherwise. In other words, it clips negative values to zero. Note that there are many other possible choices for the activation function (see figure 3.13), but the ReLU is the most commonly used and the easiest to understand.



- The model represents a family of equations that model the relationship:

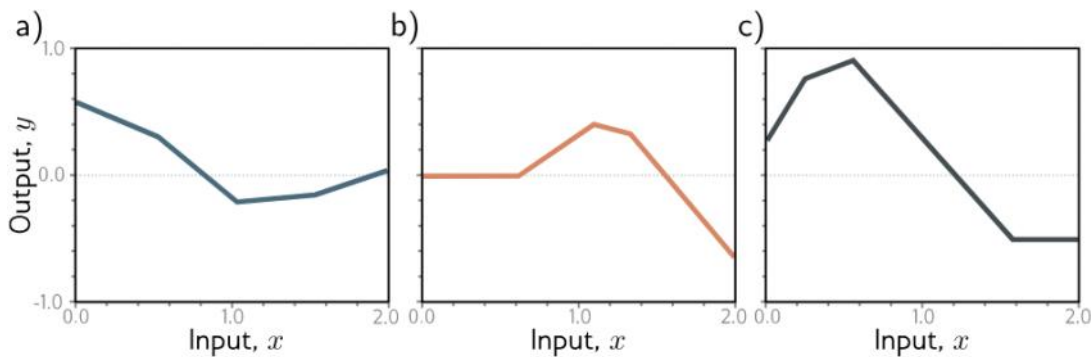


Figure 3.2 Family of functions defined by equation 3.1. a–c) Functions for three different choices of the ten parameters ϕ . In each case, the input/output relation is piecewise linear. However, the positions of the joints, the slopes of the linear regions between them, and the overall height vary.

- We can build an intuition as to why the example model describes the family it does
↳ Continuous, piecewise linear, with up to 4 linear regions.

- We can split the model into 2 parts:

1) The intermediate quantities:

$$h_1 = a[\theta_{10} + \theta_{11}x]$$

$$h_2 = a[\theta_{20} + \theta_{21}x]$$

$$h_3 = a[\theta_{30} + \theta_{31}x]$$

h_n is referred to as a hidden unit.

~

2) Compute the output by combining the hidden units unit with a linear function:

$$y = \theta_0 + \theta_1 h_1 + \theta_2 h_2 + \theta_3 h_3$$

by clipping the output by constraining the hidden units only with a linear function.

$$y = \phi_0 + \phi_1 h_1 + \phi_2 h_2 + \phi_3 h_3$$

- So each hidden unit computes a linear function $\theta_0 + \theta_1 x$. This line is clipped by ReLU, a [J].

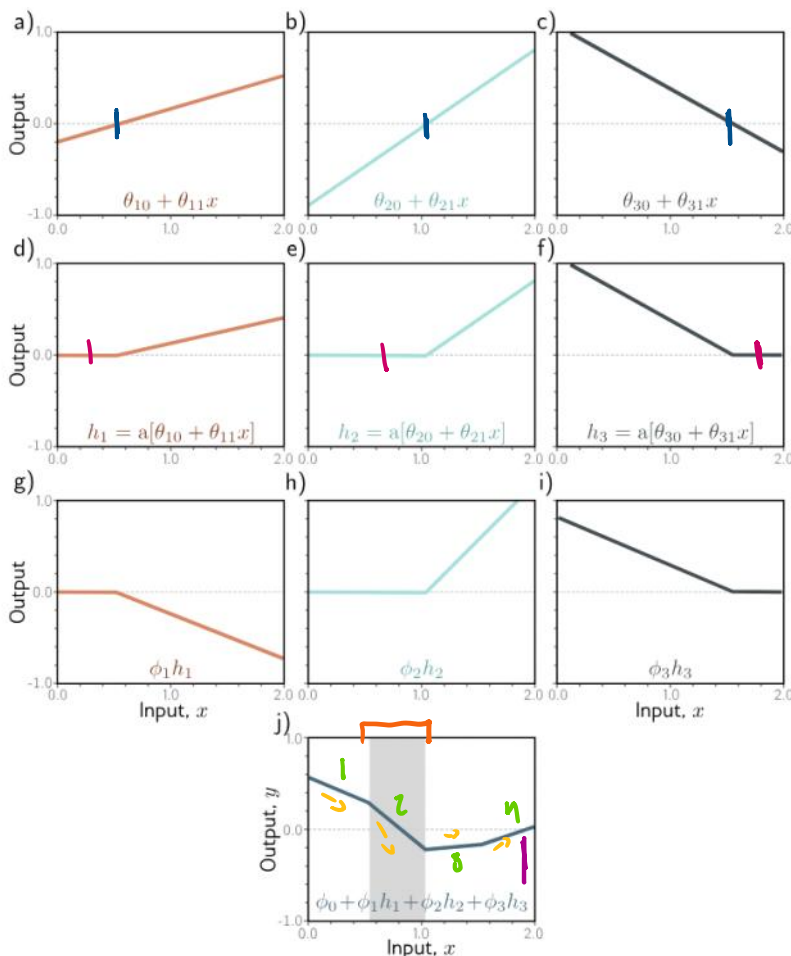


Figure 3.3 Computation for function in figure 3.2a. a-c) The input x is passed through three linear functions, each with a different y -intercept $\theta_{\bullet 0}$ and slope $\theta_{\bullet 1}$. d-f) Each line is passed through the ReLU activation function, which clips negative values to zero. g-i) The three clipped lines are then weighted (scaled) by ϕ_1 , ϕ_2 , and ϕ_3 , respectively. j) Finally, the clipped and weighted functions are summed, and an offset ϕ_0 that controls the height is added. Each of the four linear regions corresponds to a different activation pattern in the hidden units. In the shaded region, h_2 is inactive (clipped), but h_1 and h_3 are both active. (Interactive figure)

- The positions where the three lines cross 0 become the three "joints" in the final output
- ϕ_0 controls the height of the final function
- Each linear region corresponds to a different activation pattern.
- When ReLU clips a unit, we say it's inactive. Otherwise, it's active.
- The shaded region receives contributions from h_1 & h_3 (which are active), but not h_2 (inactive).
- The slope of each linear region is determined by the original slopes $\theta_{\bullet 1}$ of the active inputs for the region, & the weights $\phi_{\bullet}$.
 ↳ Ex: The slope in the shaded region is:

$$\phi_1 \theta_{11} + \phi_3 \theta_{31}$$

$$\underbrace{\phi_1 \theta_{11}}_{h_1} + \underbrace{\phi_3 \theta_{31}}_{h_3}$$

- Each hidden unit contributes a "joint" to the function.

↳ 3 hidden functions = 4 linear regions.

↳ only 3 slopes of the regions are independent.

↳ one is either 0 [if all hidden units are inactive] or sum of slopes of other regions

→ one is either 0 [if all hidden units are inactive] or sum of slopes of other regions

• We can depict the shallow neural network as sch:

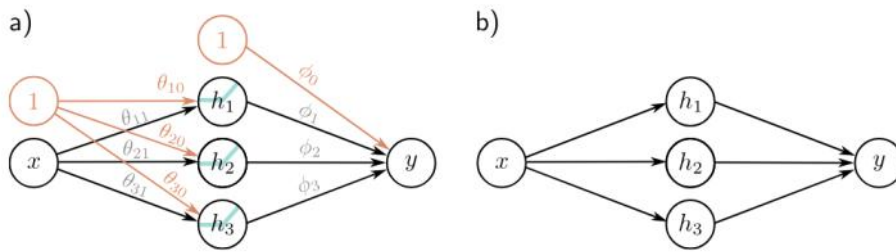


Figure 3.4 Depicting neural networks. a) The input x is on the left, the hidden units h_1, h_2 , and h_3 in the center, and the output y on the right. Computation flows from left to right. The input is used to compute the hidden units, which are combined to create the output. Each of the ten arrows represents a parameter (intercepts in orange and slopes in black). Each parameter multiplies its source and adds the result to its target. For example, we multiply the parameter ϕ_1 by source h_1 and add it to y . We introduce additional nodes containing ones (orange circles) to incorporate the offsets into this scheme, so we multiply ϕ_0 by one (with no effect) and add it to y . ReLU functions are applied at the hidden units. b) More typically, the intercepts, ReLU functions, and parameter names are omitted; this simpler depiction represents the same network.

3.2: Universal Approximation Theorem:

- The example above had 1 input, 1 output, 3 hidden units.
- We can generalize ; Consider the case with 3 hidden units, where the d^{th} unit is:

$$h_d = a[\theta_{d0} + \theta_{d1}x]$$

- These are linearly combined to give:

$$y = \phi_0 + \sum_{d=1}^D \phi_d h_d$$

- The number of hidden units gives us network capacity.
- With ReLU activation functions, the model has $D+1$ linear regions.
- With enough capacity, the shallow network can describe any continuous 1D function
- The universal approximation theorem says: For any continuous function, there exists a

- The universal approximation theorem says: For any continuous function, there exists a shallow network that can approximate this function to any specified position.

↳ Generalizes > 1D!

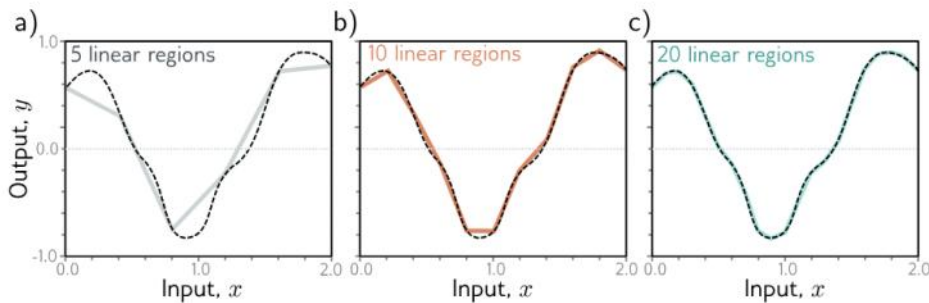


Figure 3.5 Approximation of a 1D function (dashed line) by a piecewise linear model. a-c) As the number of regions increases, the model becomes closer and closer to the continuous function. A neural network with a scalar input creates one extra linear region per hidden unit. This idea generalizes to functions in D_1 dimensions. The universal approximation theorem proves that, with enough hidden units, there exists a shallow neural network that can describe any given continuous function defined on a compact subset of $\mathbb{R}^{D_1}$ to arbitrary precision.

3.3: Multivariate Inputs & Outputs:

- We can extend the model above to predict multivariate outputs:
↳ Simply use a different linear function of the hidden units for each output.

↳ Ex:

- scalar input x
- 4 hidden units
- 2D multivariate output $y = [y_1, y_2]^T$

$$h_1 = a[\theta_{10} + \theta_{11}x]$$

$$h_2 = a[\theta_{20} + \theta_{21}x]$$

$$h_3 = a[\theta_{30} + \theta_{31}x]$$

$$h_4 = a[\theta_{40} + \theta_{41}x]$$

Then:

$$y_1 = \phi_{10} + \phi_{11}h_1 + \phi_{12}h_2 + \phi_{13}h_3 + \phi_{14}h_4$$

$$y_2 = \phi_{20} + \phi_{21}h_1 + \phi_{22}h_2 + \phi_{23}h_3 + \phi_{24}h_4$$

This is what is added

$$y_2 = \phi_{20} + \phi_{21}h_1 + \phi_{22}h_2 + \phi_{23}h_3 + \phi_{24}h_4$$

- Visually, the network looks like:

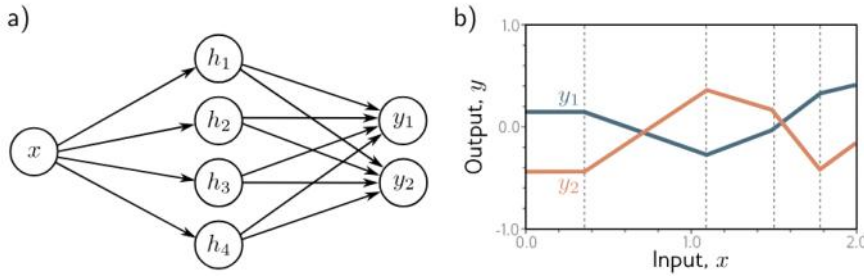


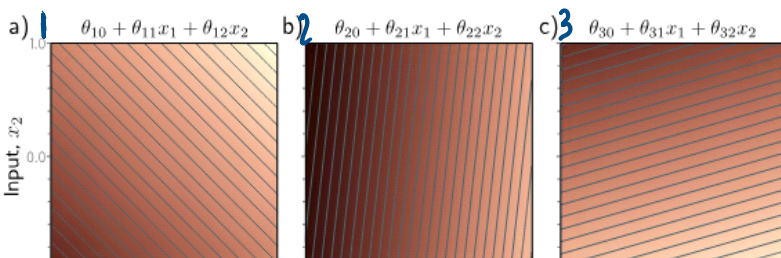
Figure 3.6 Network with one input, four hidden units, and two outputs. a) Visualization of network structure. b) This network produces two piecewise linear functions, $y_1[x]$ and $y_2[x]$. The four “joints” of these functions (at vertical dotted lines) are constrained to be in the same places since they share the same hidden units, but the slopes and overall height may differ.

- The joints are located in the same places because they share hidden units.
- To handle multivariate inputs x , we extend the relations between the input & the hidden units:

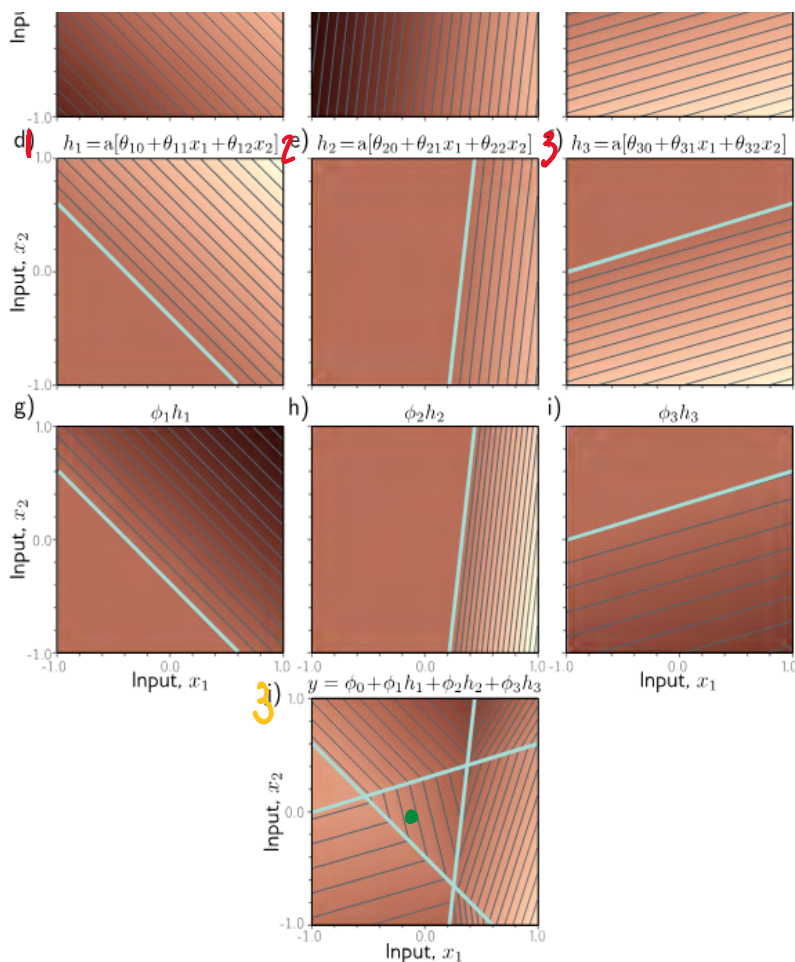
↳ A network with 2 inputs $x = [x_1, x_2]^T$ & scalar output y with 3 units is defined as:

$$\left. \begin{aligned} h_1 &= a[\theta_{10} + \theta_{11}x_1 + \theta_{12}x_2] \\ h_2 &= a[\theta_{20} + \theta_{21}x_1 + \theta_{22}x_2] \\ h_3 &= a[\theta_{30} + \theta_{31}x_1 + \theta_{32}x_2] \end{aligned} \right\} \begin{array}{l} \text{There is now} \\ \text{one slope parameter} \\ \text{per input.} \end{array}$$

$$y = \phi_0 + \phi_1h_1 + \phi_2h_2 + \phi_3h_3$$



- Each hidden unit forms an oriented linear plane in 3D space
- Each plane is clipped by ReLU.
- Each oriented plane is combined to make



- Each plane is clipped by ReLU.
- Each clipped plane is recombined to make a continuous, piecewise linear surface consisting of convex polygon regions.
- Each region corresponds to an activation pattern. The central triangle has h_1 & h_3 active, but h_2 inactive.

Figure 3.8 Processing in network with two inputs $\mathbf{x} = [x_1, x_2]^T$, three hidden units h_1, h_2, h_3 , and one output y . a–c) The input to each hidden unit is a linear function of the two inputs, which corresponds to an oriented plane. Brightness indicates function output. For example, in panel (a), the brightness represents $\theta_{10} + \theta_{11}x_1 + \theta_{12}x_2$. Thin lines are contours. d–f) Each plane is clipped by the ReLU activation function (cyan lines are equivalent to “joints” in figures 3.3d–f). g–i) The clipped planes are then weighted, and j) summed together with an offset that determines the overall height of the surface. The result is a continuous surface made up of convex piecewise linear polygonal regions. (Interactive figure)

This work is subject to a Creative Commons CC-BY-NC-ND license. (C) MIT Press.

- When there are more than two inputs, it gets hard to visualize, but the interpretation is the same.
 - ↳ output is still continuous, piecewise linear surface, ; the linear regions are convex polytopes.
- As input dimension grows, the # of linear regions increases rapidly.
 - ↳ If # of hidden units = # of inputs:
 - 2D = 4 regions
 - 3D = 8 regions
 - D dimensions = 2^D regions.
 - ↳ Shallow networks usually have more hidden units than inputs, so we typically create $> 2^D$ regions!

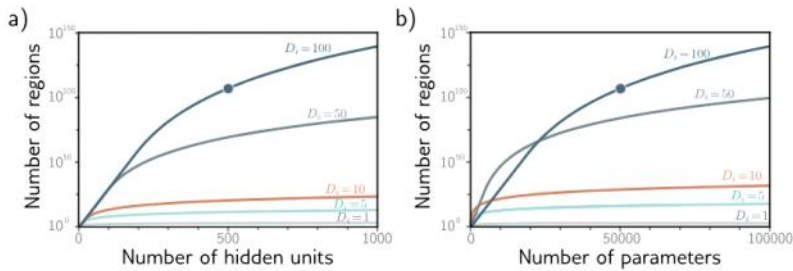


Figure 3.9 Linear regions vs. hidden units. a) Maximum possible regions as a function of the number of hidden units for five different input dimensions $D_i = \{1, 5, 10, 50, 100\}$. The number of regions increases rapidly in high dimensions; with $D = 500$ units and input size $D_i = 100$, there can be greater than 10^{107} regions (solid circle). b) The same data are plotted as a function of the number of parameters. The solid circle represents the same model as in panel (a) with $D = 500$ hidden units. This network has 51,001 parameters and would be considered very small by modern standards.

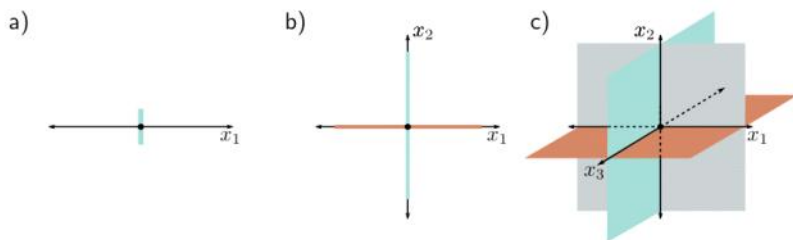


Figure 3.10 Number of linear regions vs. input dimensions. a) With a single input dimension, a model with one hidden unit creates one joint, which divides the axis into two linear regions. b) With two input dimensions, a model with two hidden units can divide the input space using two lines (here aligned with axes) to create four regions. c) With three input dimensions, a model with three hidden units can divide the input space using three planes (again aligned with axes) to create eight regions. Continuing this argument, it follows that a model with D_i input dimensions and D_i hidden units can divide the input space with D_i hyperplanes to create 2^{D_i} linear regions.

3.9: Shallow Neural Networks: General case:

- We define a general evaluation for the shallow neural network $y = f[x, \theta]$ that maps a multi-dimensional input $x \in \mathbb{R}^{D_i}$ to a multi-dimensional output $y \in \mathbb{R}^{D_o}$ with $h \in \mathbb{R}^D$ hidden units:

Each hidden unit is computed as:

$$h_d = a \left[\theta_{d0} + \sum_{i=1}^{D_i} \theta_{di} x_i \right]$$

The hidden units are combined linearly to create the output:

$$y_j = \beta_{j0} + \sum_{i=1}^D \beta_{ji} h_i$$

- $a[\cdot]$ is a nonlinear activation function.
- The model has $\theta = \{\theta_{d0}, \theta_{di}\}$ parameters

The activation function allows the model to describe non-linear relationships between input & output. **IT MUST BE NON-LINEAR**

Otherwise the resulting mapping is linear...

the model has $\mu = (w_{00}, w_{01})$ parameters

is linear...

- The most common activation function is ReLU, because it is easily interpretable.
 - ↳ Each convex polytope contains a different linear function
 - ↳ The polytopes are the same for each output, but the linear functions they contain can differ.

3.5: Terminology:

- Neural nets are composed of layers:
 - ↳ Input layer
 - ↳ Hidden layer(s)
 - ↳ Output layer
- Hidden units can be referred to as neurons.
- During inference, the values of the inputs before the activation function is applied are called pre-activations.
- The value after the activation function are called activations.
- Networks with at least 1 hidden layer are called MLPs. ↳ Multi-layer Perceptron
- Networks with 1 hidden layer are called shallow neural networks.
- Networks with > 1 hidden layers are called deep neural networks.
- Networks where the connections form an acyclic graph are called feed-forward networks.
 - ↳ A graph with no loops.
- If every element in one layer connects to every element in the next, the network is fully connected.
- Connections refer to the slope parameters in the underlying equations; are called network weights.
- The offset parameters $[b_{00}, b_{01}]$ are called biases.

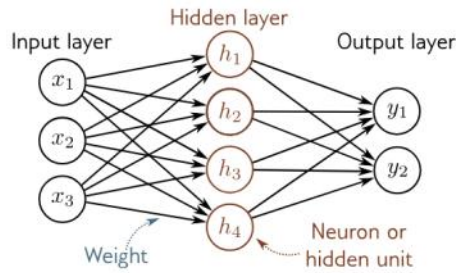


Figure 3.12 Terminology. A shallow network consists of an input layer, a hidden layer, and an output layer. Each layer is connected to the next by forward connections (arrows). For this reason, these models are referred to as feed-forward networks. When every variable in one layer connects to every variable in the next, we call this a fully connected network. Each connection represents a slope parameter in the underlying equation, and these parameters are termed weights. The variables in the hidden layer are termed neurons or hidden units. The values feeding into the hidden units are termed pre-activations, and the values at the hidden units (i.e., after the ReLU function is applied) are termed activations.

3.6: Summary:

- Shallow networks make predictions (y) based on inputs (x) by dividing the input space into a continuous surface of piecewise linear regions.
- Can approximate any continuous function to arbitrary precision given enough hidden units.